

Modül 29

Advanced TypeScript

Type-Level Programming

Modül:	29 / 30
Ders Sayısı:	8 ders
Seviye:	İleri
Versiyon:	Angular v21

Hazırlayan: Claude • Türkçe Angular v21 Eğitimi

📖 Modül 29'a Hoş Geldin

TypeScript'in tam gücünü kullanmak. Generics, utility types, conditional types, template literal types, ve mapped types ile type-safe API'lar yazmak. Modern Angular kodunda bu pattern'leri her gün göreceksin.

Öğrenecekler:

- ✓ Generics derinlemesine
- ✓ Utility types (Partial, Pick, Omit, Required)
- ✓ Conditional types
- ✓ Mapped types
- ✓ Template literal types
- ✓ Type guards ve narrowing
- ✓ Discriminated unions
- ✓ Branded types ve nominal typing

Generics Derinlemesine

Tip parametreleri ile reusable kod.

□ Temel Generic Function

```
// İlk gördüğümüz
function identity<T>(value: T): T {
  return value;
}

const x = identity<string>('hello'); // x: string
const y = identity(42);               // y: number (inferred)

// Generic constraint
function getProperty<T, K extends keyof T>(obj: T, key: K): T[K] {
  return obj[key];
}

const user = { name: 'John', age: 30 };
const name = getProperty(user, 'name'); // string
const age = getProperty(user, 'age');   // number
// getProperty(user, 'email'); // ❌ Compile error
```

□ Generic Class

```

class Repository<T extends { id: string }> {
  private items = new Map<string, T>();

  add(item: T): void {
    this.items.set(item.id, item);
  }

  get(id: string): T | undefined {
    return this.items.get(id);
  }

  getAll(): T[] {
    return Array.from(this.items.values());
  }

  filter(predicate: (item: T) => boolean): T[] {
    return this.getAll().filter(predicate);
  }
}

// Kullanım
interface User { id: string; name: string; email: string; }
const users = new Repository<User>();
users.add({ id: '1', name: 'John', email: 'j@x.com' });

```

□ Generic Constraints

```

// extends keyof
function pluck<T, K extends keyof T>(items: T[], key: K): T[K][] {
  return items.map(item => item[key]);
}

const names = pluck(users, 'name'); // string[]
const ages = pluck(users, 'age');    // number[]

// Default type
interface ApiResponse<T = unknown> {
  data: T;
  status: number;
}

const resp1: ApiResponse = { data: 'anything', status: 200 };
const resp2: ApiResponse<User> = { data: { name: 'X' } as User, status: 200 };

// Multiple constraints
interface Named { name: string; }
interface Aged { age: number; }

function describe<T extends Named & Aged>(person: T): string {
  return `${person.name} is ${person.age}`;
}

```

Utility Types

Built-in type helper'lar.

□ Partial

```
interface User {
  id: string;
  name: string;
  email: string;
  age: number;
}

// Tüm property'ler optional
type UserUpdate = Partial<User>;
// { id?: string; name?: string; email?: string; age?: number; }

function updateUser(id: string, updates: Partial<User>) {
  // Sadece bazı field'ları güncelle
}

updateUser('123', { name: 'New Name' }); // ✓
updateUser('123', { name: 'X', email: 'x@x.com' }); // ✓
```

✂ Pick ve Omit

```
// Pick – sadece istenen field'lar
type UserSummary = Pick<User, 'id' | 'name'>;
// { id: string; name: string; }

// Omit – istenmeyen field'ları çıkar
type CreateUserDto = Omit<User, 'id'>;
// { name: string; email: string; age: number; }

function createUser(data: CreateUserDto): User {
  return { ...data, id: crypto.randomUUID() };
}
```

□ Required ve Readonly

```
// Required – tüm property'leri zorunlu yap
type StrictUser = Required<Partial<User>>;
// Aynı User (gereksiz örnek ama anlatım için)

// Readonly – değiştirilemez
type FrozenUser = Readonly<User>;
const user: FrozenUser = { id: '1', name: 'J', email: 'j@x.com', age: 30 };
// user.name = 'X'; // ❌ Compile error
```

❑ Record

```
// Key-value mapping
type RolePermissions = Record<'admin' | 'user' | 'guest', string[]>;

const permissions: RolePermissions = {
  admin: ['read', 'write', 'delete'],
  user: ['read', 'write'],
  guest: ['read']
};

// Dynamic key
type Translations = Record<string, string>;
const t: Translations = { hello: 'Merhaba', bye: 'Hoşçakal' };
```

❑ ReturnType, Parameters, Awaited

```
function getUser(id: string): { name: string; age: number } {
  return { name: 'X', age: 30 };
}

type UserReturn = ReturnType<typeof getUser>;
// { name: string; age: number; }

type GetUserParams = Parameters<typeof getUser>;
// [id: string]

// Promise unwrap
async function fetchUser(): Promise<User> { /* ... */ }

type FetchedUser = Awaited<ReturnType<typeof fetchUser>>;
// User (Promise unwrapped)
```

Conditional Types

T extends U ? X : Y — type-level if statement.

□ Temel Conditional

```
type IsString<T> = T extends string ? true : false;

type A = IsString<'hello'>; // true
type B = IsString<42>;      // false

// Practical
type NonNull<T> = T extends null | undefined ? never : T;

type X = NonNull<string | null>; // string
type Y = NonNull<number | undefined>; // number
```

□ Infer Keyword

```
// Function'ın return type'ını çek
type GetReturn<T> = T extends (...args: any[]) => infer R ? R : never;

type X = GetReturn<() => string>; // string
type Y = GetReturn<(x: number) => number[]>; // number[]

// Promise unwrap
type Unpromise<T> = T extends Promise<infer U> ? U : T;

type A = Unpromise<Promise<string>>; // string
type B = Unpromise<number>; // number (zaten Promise değil)

// Array element type
type ElementType<T> = T extends (infer U)[] ? U : never;
type StringElement = ElementType<string[]>; // string
```

⚡ Practical Use Cases

```
// Function olup olmadığına göre farklı işle
type Process<T> = T extends (...args: any[]) => any
  ? 'function'
  : T extends any[]
  ? 'array'
  : T extends object
  ? 'object'
  : 'primitive';

type T1 = Process<() => void>;           // 'function'
type T2 = Process<string[]>;           // 'array'
type T3 = Process<{ x: 1 }>;           // 'object'
type T4 = Process<string>;             // 'primitive' fallback
```

Mapped Types

Object'in tüm property'lerini dönüştürmek.

□ Temel Mapped Type

```
// Tüm property'leri optional yap
type MyPartial<T> = {
  [K in keyof T]?: T[K];
};

// Tüm property'leri readonly yap
type MyReadOnly<T> = {
  readonly [K in keyof T]: T[K];
};

// Tüm property'leri nullable yap
type Nullable<T> = {
  [K in keyof T]: T[K] | null;
};

interface User { id: string; name: string; age: number; }
type NullableUser = Nullable<User>;
// { id: string | null; name: string | null; age: number | null; }
```

⚙ Modifier'lar (+, -)

```
// + ve - ile modifier ekle/çıkar

// readonly ekle
type ReadOnly<T> = {
  +readonly [K in keyof T]: T[K];
};

// readonly kaldır
type Mutable<T> = {
  -readonly [K in keyof T]: T[K];
};

// optional kaldır
type RequiredAll<T> = {
  [K in keyof T]-?: T[K];
};

interface Config {
  readonly id: string;
  name?: string;
}

type EditableConfig = Mutable<RequiredAll<Config>>;
// { id: string; name: string; }
```

□ Key Renaming (as)

```
// Getter'lar oluştur
type Getters<T> = {
  [K in keyof T as `get${Capitalize<string & K>}`]: () => T[K];
};

interface User { id: string; name: string; age: number; }

type UserGetters = Getters<User>;
// {
//   getId: () => string;
//   getName: () => string;
//   getAge: () => number;
// }

// Filter – sadece string property'ler
type StringProps<T> = {
  [K in keyof T as T[K] extends string ? K : never]: T[K];
};

type UserStrings = StringProps<User>; // { id: string; name: string; }
```

Template Literal Types

String type'larıyla type-level magic.

□ Temel Template Literal

```
type Greeting = `Hello, ${string}!`;

const g1: Greeting = 'Hello, World!'; // ✓
const g2: Greeting = 'Hi';           // □

// Union ile birleşim
type Color = 'red' | 'green' | 'blue';
type Size = 'sm' | 'md' | 'lg';

type ButtonClass = `btn-${Color}-${Size}`;
// "btn-red-sm" | "btn-red-md" | ... | "btn-blue-lg"
// 9 farklı string
```

□ Practical: Event Names

```
type Mouse = 'click' | 'hover';
type Keyboard = 'keydown' | 'keyup';

type EventName = `on${Capitalize<Mouse | Keyboard>}`;
// "onClick" | "onHover" | "onKeyDown" | "onKeyUp"

// CRUD action namespacing
type Resource = 'users' | 'posts' | 'comments';
type Action = 'create' | 'read' | 'update' | 'delete';

type Permission = `${Resource}:${Action}`;
// "users:create" | "users:read" | ... 12 kombinasyon

const userCanCreate: Permission = 'users:create'; // ✓
const invalid: Permission = 'profile:read'; // □
```

↩ String Manipulation

```
// Built-in string utility types
type A = Uppercase<'hello'>;    // 'HELLO'
type B = Lowercase<'HELLO'>;    // 'hello'
type C = Capitalize<'hello'>;   // 'Hello'
type D = Uncapitalize<'Hello'>; // 'hello'

// Object property'lerden type üret
interface Config {
  apiUrl: string;
  timeout: number;
  retries: number;
}

type EnvVar = `${Uppercase<string & keyof Config>}_ENV`;
// "APIURL_ENV" | "TIMEOUT_ENV" | "RETRIES_ENV"
```

Type Guards & Narrowing

Runtime'da type narrowing.

▣ typeof Guard

```
function format(value: string | number): string {  
  if (typeof value === 'string') {  
    return value.toUpperCase(); // value: string  
  }  
  return value.toFixed(2); // value: number  
}
```

▣ instanceof Guard

```
class HttpError extends Error {  
  constructor(public status: number, msg: string) {  
    super(msg);  
  }  
}  
  
class ValidationError extends Error {  
  constructor(public fields: string[], msg: string) {  
    super(msg);  
  }  
}  
  
function handleError(err: unknown) {  
  if (err instanceof HttpError) {  
    console.log(err.status); // err: HttpError  
  } else if (err instanceof ValidationError) {  
    console.log(err.fields); // err: ValidationError  
  } else if (err instanceof Error) {  
    console.log(err.message); // err: Error  
  }  
}
```

▣ in Operator

```

interface Bird { fly(): void; }
interface Fish { swim(): void; }

function move(animal: Bird | Fish) {
  if ('fly' in animal) {
    animal.fly(); // animal: Bird
  } else {
    animal.swim(); // animal: Fish
  }
}

```

□ Custom Type Predicate

```

// "is" return type – type guard function
function isUser(value: unknown): value is User {
  return (
    typeof value === 'object' &&
    value !== null &&
    'id' in value &&
    'email' in value &&
    typeof (value as any).email === 'string'
  );
}

const data: unknown = await fetchData();

if (isUser(data)) {
  console.log(data.email); // data: User
}

// Array filter
const items: unknown[] = [...];
const users = items.filter(isUser); // User[]

```

□ Assertion Function

```

// "asserts" – return ederse, throw etmedikse type true
function assertIsUser(value: unknown): asserts value is User {
  if (!isUser(value)) {
    throw new Error('Not a User');
  }
}

function processData(data: unknown) {
  assertIsUser(data);
  console.log(data.email); // data: User (assert sonrası)
}

```

Discriminated Unions

Type-safe state machines.

□ Pattern

```
// State machine – her state'in unique "type" field'ı var
type AsyncState<T> =
  | { type: 'idle' }
  | { type: 'loading' }
  | { type: 'success'; data: T }
  | { type: 'error'; error: Error };

function render<T>(state: AsyncState<T>) {
  switch (state.type) {
    case 'idle':
      return 'Bekliyor';
    case 'loading':
      return 'Yükleniyor...';
    case 'success':
      return state.data; // ✓ data sadece success'te var
    case 'error':
      return state.error.message; // ✓ error sadece error'da
  }
}
```

□ Exhaustive Check

```

type Action =
  | { type: 'INCREMENT' }
  | { type: 'DECREMENT' }
  | { type: 'SET'; payload: number };

function reducer(state: number, action: Action): number {
  switch (action.type) {
    case 'INCREMENT':
      return state + 1;
    case 'DECREMENT':
      return state - 1;
    case 'SET':
      return action.payload;
    default:
      // never type – eğer yeni action eklenirse compile error
      const _exhaustive: never = action;
      return state;
  }
}

```

□ Component Pattern

```

@Component({
  template: `
    @switch (state().type) {
      @case ('idle') {
        <p>Hazır</p>
      }
      @case ('loading') {
        <app-spinner />
      }
      @case ('success') {
        <app-data-view [data]="state().data" />
        <!-- TypeScript bilir: data sadece success'te var -->
      }
      @case ('error') {
        <p>Hata: {{ state().error.message }}</p>
      }
    }
  `
})
export class DataComponent<T> {
  state = signal<AsyncState<T>>({ type: 'idle' });
}

```

Branded Types — Nominal Typing

Aynı primitive'in farklı kullanım amaçlarını ayırt etmek.

❑ Problem

```
// String iki farklı şey için kullanılıyor
function getUser(userId: string) { /* ... */ }

const email = 'j@example.com';
getUser(email); // ❌ Anlamsız ama TypeScript hata vermez!
```

❑ Branded Type Çözümü

```
// Brand – phantom type
type Brand<T, B> = T & { __brand: B };

type UserId = Brand<string, 'UserId'>;
type Email = Brand<string, 'Email'>;
type Password = Brand<string, 'Password'>;

// Constructor function'lar
function createUserId(value: string): UserId {
  // Validation eklenebilir
  return value as UserId;
}

function createEmail(value: string): Email {
  if (!value.includes('@')) throw new Error('Invalid email');
  return value as Email;
}

// Kullanım
function getUser(id: UserId) { /* ... */ }

const id = createUserId('123');
const email = createEmail('j@example.com');

getUser(id); // ✓
getUser(email); // ❌ Compile error!
getUser('123'); // ❌ Compile error (raw string)
```

❑ Practical Examples

```
// ID Brands
type ProjectId = Brand<string, 'ProjectId'>;
type UserId = Brand<string, 'UserId'>;
type RoleId = Brand<string, 'RoleId'>;

// Validated types
type ValidatedEmail = Brand<string, 'ValidatedEmail'>;
type HashedPassword = Brand<string, 'HashedPassword'>;

function hashPassword(plain: string): HashedPassword {
  return bcrypt.hash(plain) as HashedPassword;
}

function login(email: ValidatedEmail, password: HashedPassword) {
  // Tip ile garantili: validated email, hashed password
}

// Bytes vs MB vs GB
type Bytes = Brand<number, 'Bytes'>;
type Megabytes = Brand<number, 'Megabytes'>;

function toMB(bytes: Bytes): Megabytes {
  return (bytes / (1024 * 1024)) as Megabytes;
}
```

□ Best Practices

- ✓ Public ID'ler için brand kullan (ProductId, UserId)
- ✓ Validated data için (ValidEmail, ValidUrl)
- ✓ Sensitive data için (HashedPassword, EncryptedToken)
- ✓ Unit conversions için (Meters, Feet, Bytes)

Modül 29 Özeti

TypeScript'in gerçek gücünü keşfettin. Type-level programming ile compile-time'da bug'ları yakalayan, IDE'nin sana yardım eden, refactoring'i kolaylaştıran kod yazabilirsin.

Neler Öğrendin?

- ✓ Generics derinlemesine (constraint, default, multiple)
- ✓ Utility types (Partial, Pick, Omit, Required, Record, ReturnType)
- ✓ Conditional types ve infer
- ✓ Mapped types ve key renaming
- ✓ Template literal types
- ✓ Type guards (typeof, instanceof, in, is, asserts)
- ✓ Discriminated unions (state machines)
- ✓ Branded types (nominal typing)

Sıradaki Modül

Modül 30 — Mimari Best Practices. Bu eğitimin son modülü. Tüm öğrendiklerini birleştirip büyük projeler için mimari kararları öğreneceğiz.